

레벨 2 인터뷰

의사결정의 핵심 기준점

- 이론과 실제의 타협
 - 이론과 원칙을 언제 따르고 언제 버릴지는 상황과 조건, 비즈니스 요구사항에 따라 결정된다.
 - 요구사항 충족을 최우선 목표로 설정하고, 달성 이후에 리팩토링을 고려한다.
 - 책임 분배와 역할 응집을 통한 유지보수성 향상을 기본 설계 방향으로 설정한다.
 - 객체 지향적 구조 위에 절차 지향적 흐름을 설계한다.

레벨 1부터 쌓아올린 고민의 과정이 있었기에 때론 기준을 잊거나 잃었다가도 리뷰어의 지적, 피드백을 통해 제자리를 찾아갈 수 있었다. 아래는 적용 사례 세 가지이다.

사례 1·이론과 타협 — 참조·조립·순번의 실용적 예외

상황 도메인을 DDD 정의대로 식별자 참조로 설계할 것인가? **결정** 식별자 참조를 버리고, 객체 기반 참조를 선택. **이유** 이론에 집착하기보다 객체가 스스로 책임을 수행하도록 설계하는 것이 올바른 객체 지향이다.

식별자로 끊으면 **Reservation** 이 행위할 때마다 **Time**·**Theme** 를 다시 조회해 조립해야 한다. 명확한 애그리거트 루트가 존재하지 않는 현 구조에서 이론적 정의보다 객체의 자율성을 우선했다.

같은 기준으로, 도메인 로직의 DB 유출도 허용했다. **리소스**·**도메인**·**스키마** 가 1:1:1로 대응하지 않는 한, 조립에는 도메인 로직이 녹아들 수 있다. 도메인 조립, 예약 가능 시간 계산, 대기 순번 산출을 자바가 아닌 조인·집계 쿼리로 **DB** 에 맡겼다. **N+1** 을 피하기 위해 도메인 로직이 쿼리로 새어든 **실용적 예외** 임을 인정한다.

결점 불변 객체로 설계할 수 있었으나 이론에 집착해 가변으로 설계했다. 불변 객체의 장점을 학습했음에도, **DDD** 의 엔티티 정의에 매였기 때문이었다. "JDBC 환경에서 가변 엔티티는 변경 시점 추적 어렵다"는 리뷰를 받아 모순점을 인지하고 수정했다.

사례 2·시도하고 확인하고 결정하기 — 동시성과 계층적 락

상황 동시 취소 등의 경합을 어떻게 막을 것인가? **결정** 계층적 락을 시도했으나 철회하고, **UNIQUE** 제약으로 수습했다. **이유** 계층적 락의 전제인 애그리거트 루트 구조가 현 도메인에 적합하지 않았다.

동시성 제어의 방법으로 계층적 락을 선택하고, **Slot**→**Session** 을 애그리거트 루트로 활용하려 했다. 코치 피드백: "지금 구조라면 오히려 예약이 애그리거트 루트인데 왜 슬롯을 루트로 보는가". 리뷰어 피드백: "별도 테이블과 생명주기 공유라는 두 방향을 동시에 채택해 복잡하다".

다시 고민해 보았을 때, 세션과 예약/대기는 종속된 생명주기가 아닌 독립된 생명주기라 판단했다. 이론에 구조를 끼워맞췄기에 막히는 부분, 역전된 구조가 신호로 나타났고 피드백이 결정타였다. 적절한 구조였던 스키마와 도메인 객체의 분리는 남기고, 애그리거트 루트라는 개념의 도입만 철회

했다. 스스로의 기준을 위배했지만, 그렇기에 기준의 중요성을 자각할 수 있었고, 기준점을 명확히 할 수 있었다.

사례 3·수평적 구조와 수직적 흐름 — 트랜잭션 경계와 예외

상황 수정된 구조에서 트랜잭션 경계와 예외를 무엇으로 정할 것인가? **결정** 수평적 계층 구조가 아니라 수직적 실행 흐름을 기준으로 정한다. **이유** 작업의 경계는 테이블의 차이가 아닌 비즈니스의 정합성이다.

예약 취소만 성공하고 대기의 승격이 발생하지 않으면 "예약 없는 대기"가 노출된다. 예약이 존재해야만 대기를 생성할 수 있다는 비즈니스 요구사항에 따라, 두 변경은 하나의 결과로 묶인다. 단순히 여러 테이블을 변경한다는 이유로 하나의 트랜잭션으로 묶지 않는다.

트랜잭션 경계는 실행 흐름이 정한다. 부분 실패가 사용자에게 비정합 상태를 노출할 수 있는 작업들은 하나의 트랜잭션으로 묶는다.

UNIQUE(session_id) 제약조건이 동시 취소의 최종 방어선 역할을 하듯 계층은 책임을 수평으로 나누고, 흐름은 작업을 수직으로 묶는다.

예외 또한 수직적 흐름의 측면에서 관리한다. **DTO** 처럼 예외도 발생한 계층에 종속되는지, 계층별로 관리하는 것이 적절할지 묻자 "계층별로 나누는 것은 추천하지 않지만, 왜 그렇게 생각했는지 먼저 설명해달라"는 리뷰를 받았다. 설계한 구조에 따라 특정 예외들을 특정 계층에 고정할 순 있었지만, "예약을 찾을 수 없다"는 실패는 서비스에서 터지든 레포지토리에서 터지든 같은 의미였다.

실패는 수직적 흐름에 종속된 개념이므로, 발생 계층이 아니라 실패의 의미로 관리한다.

학습 과정

기준을 수립하고 강화하는 과정에 다양한 학습, 여러 피드백이 큰 도움이 되었다.

- **CQS**의 명령/조회 분리에서 출발해 Stack pop과 AUTO_INCREMENT 반례로 **실용적 예외**의 필요성을 정리했다.
- **N+1** 문제의 발생 지점을 확인한 뒤 조인과 조립의 책임 분배로 정리했다.
- **DDD**의 목적을 "소프트웨어 중심부의 복잡성 통제"로 정리하되 실제 구현에 적용하는 것은 선을 그었다.
- **Fake**와 실제 구현의 계약 불일치를 겪으며 테스트의 필요성과 테스트 유지보수의 중요성을 인식했다.

규칙과 원칙, 이론은 **정답**이 아니라 목적 달성에 필요한 **수단**이다.